REVERSE ENGINEERING AND ANALYSIS OF MALICIOUS BINARY EXECUTABLES

BY

DAMOAH BASHIRU

ABSTRACT

Malware continues to evolve as one of the most persistent threats in modern cybersecurity, with attackers increasingly relying on obfuscation and packing techniques to evade detection. This report presents a detailed reverse engineering analysis of a malicious binary executable identified as a LockScreen ransomware sample, compiled using the AutoIt scripting language and protected with UPX packing.

The analysis was conducted entirely within an isolated virtual machine environment using three industry-standard tools, PEStudio, Ghidra, and x32dbg, following a structured methodology that combined static and dynamic analysis techniques. Beginning with binary structure examination, the investigation uncovered a heavily obfuscated PE32 executable with 156 flagged imports, self-modifying code sections, and an entropy value indicative of packed content. Unpacking the binary with UPX revealed the full extent of its capabilities, including process injection routines, network communication functions, registry manipulation, and aggressive screen-locking behavior.

Decompilation in Ghidra exposed the AutoIt runtime engine embedded within the binary, while debugging sessions in x32dbg confirmed the malware's ability to load critical system libraries, hook input functions, and deploy its lockscreen payload before conventional API breakpoints could intercept it. Extracted indicators of compromise include references to all major registry hives, system reconnaissance variables, ICMP-based network probing, and privilege escalation tokens such as SeDebugPrivilege and SeShutdownPrivilege.

The findings of this report contribute to a broader understanding of how commodity malware families leverage scripting languages and packers to achieve persistence and resistance to analysis, and demonstrate the value of combining multiple reverse engineering approaches when investigating obfuscated threats.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
AV	Antivirus
AU3	AutoIt Version 3
C2	Command and Control
COM	Component Object Model
CPU	Central Processing Unit
DLL	Dynamic Link Library
EIP	Extended Instruction Pointer
FTP	File Transfer Protocol
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IAT	Import Address Table
ICMP	Internet Control Message Protocol
IOC	Indicator of Compromise
MD5	Message Digest Algorithm 5
OS	Operating System
PE	Portable Executable
PID	Process Identifier
RAM	Random Access Memory
SHA	Secure Hash Algorithm
UPX	Ultimate Packer for eXecutables
VM	Virtual Machine
WinAPI	Windows Application Programming Interface

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Malware analysis is a critical discipline within the field of cybersecurity, enabling analysts and researchers to understand the internal mechanics of malicious software and develop effective countermeasures. Among the various approaches to malware analysis, reverse engineering stands out as one of the most thorough and revealing techniques. Unlike behavioral monitoring, which only observes what a program does at runtime, reverse engineering allows the analyst to examine the actual code and logic embedded within a binary, uncovering hidden functionality that may not be immediately visible through observation alone.

This report presents a detailed reverse engineering analysis of a malicious binary executable, identified and referred to throughout this report as ransomware.exe. The sample was sourced from a local malware repository and confirmed as malicious by 57 out of 72 security vendors on VirusTotal, with a popular threat label of trojan.lockscreen/autoit. The binary belongs to the LockScreen ransomware family and was compiled using the AutoIt version 3 scripting language, a legitimate automation tool that has been widely abused by malware authors due to its ability to compile scripts into standalone executables that are difficult to analyze without the right tools and methodology.

1.2 OBJECTIVES

The primary objective of this assignment is to train students to analyze and understand the internal structure and functionality of malware by examining its binary code. Specifically this report aims to achieve the following:

- To examine the binary structure of the malware sample using PEStudio and identify its file format, entry point, imported libraries, and suspicious API calls.
- To decompile the malware binary using Ghidra and identify functions responsible for network communication, persistence mechanisms, and data collection.

- To observe malware behavior at runtime using x32dbg by setting breakpoints, stepping through execution, and monitoring memory and register changes.
- To determine the functional capabilities of the malware including screen locking, system reconnaissance, privilege escalation, and network activity.
- To extract indicators of compromise including hardcoded strings, registry keys, privilege tokens, and network-related artifacts for use in threat intelligence.

1.3 SAMPLE DECLARATION

The malware sample analyzed in this report is declared in the table below. All hash values were verified using the Windows certutil utility within the isolated virtual machine environment prior to analysis.

Table 1.1 Malware Sample Declaration

Property	**Value**
File Name	ransomware.exe
SHA256	265041a4e943debd8b6b147085cb8 549be110facde2288021e90ae65e87be235
MD5	63bab74409c514ed8548b1f33d0acedc
SHA1	abc6bb8dd01fa83d7fd92182601b868d2b6dd1ea
File Type	PE32 Executable, 32-bit, GUI
Malware Family	LockScreen Ransomware (AutoIt)
Source	Local Malware Repository
VirusTotal Detection	57 out of 72

1.4 TOOLS USED

The following tools were used throughout the analysis. All tools were installed and operated exclusively within an isolated Windows 10 virtual machine to ensure the safety of the host system.

- **PEStudio** was used for static binary structure analysis, including examination of the PE header, imported libraries, flagged API calls, and embedded strings.
- **Ghidra** was used for decompilation of the unpacked binary, function identification, and code-level analysis of the malware logic.

- **UPX** was used to unpack the binary prior to deeper analysis, as the sample was confirmed to be compressed using the UPX packer.

1.5 REPORT ORGANISATION

The remainder of this report is organised as follows. Chapter 2 presents the binary structure analysis conducted using PEStudio, covering the PE header, imported libraries, suspicious API calls, and string extraction findings. Chapter 3 presents the decompilation findings from Ghidra and the debugging observations recorded using x32dbg. Chapter 4 presents the malware capability analysis and the full list of extracted indicators of compromise identified throughout the investigation.

CHAPTER 2

LITERATURE REVIEW

2.1 MALWARE ANALYSIS AND REVERSE ENGINEERING

Malware analysis has become an indispensable component of modern cybersecurity practice. As malicious software grows in complexity and volume, the ability to dissect and understand its internal workings has become increasingly valuable to defenders, incident responders, and threat intelligence teams. Broadly speaking, malware analysis can be divided into two complementary approaches, static analysis and dynamic analysis, each offering distinct advantages depending on the nature of the sample being investigated.

Static analysis involves examining a binary without executing it, relying on tools that inspect the file structure, imported libraries, embedded strings, and disassembled code. Dynamic analysis, on the other hand, involves running the malware in a controlled environment and observing its behavior in real time. Research published in Computers and Security has consistently shown that combining both approaches produces a more complete and accurate picture of malware behavior than either method alone. Reverse engineering sits at the intersection of these two approaches, enabling analysts to go beyond surface level observations and examine the actual logic embedded within the binary code.

2.2 AUTOIT AS A MALWARE DEVELOPMENT PLATFORM

AutoIt is a freeware scripting language originally designed for automating Windows graphical user interface interactions. Its simplicity, flexibility, and ability to compile scripts into standalone executable files have made it an attractive tool not only for system administrators but also for malware authors. AutoIt compiled binaries embed a full runtime interpreter within the executable, making them significantly more difficult to analyze than traditionally compiled programs.

Several malware families have been documented using AutoIt as their development platform, ranging from simple downloaders and keyloggers to more complex remote access trojans and ransomware. The LockScreen family, to which the sample analyzed in this report belongs, is one such example. AutoIt based malware typically evades signature based detection by

obfuscating the embedded script and relying on the legitimacy of the AutoIt runtime to bypass antivirus heuristics. Studies examining AutoIt malware have noted that the compiled binary often contains thousands of internal strings and functions belonging to the AutoIt engine itself, making it challenging to isolate the malicious logic without first unpacking and filtering the binary.

2.3 EXECUTABLE PACKING AND OBFUSCATION

Packing is a common technique used by malware authors to compress and obfuscate a binary executable, making it harder for security tools to analyze the file statically. The UPX packer, which was used to protect the sample examined in this report, is one of the most widely used packers in both legitimate software and malware. When a UPX packed binary is executed, it first runs an unpacking stub that decompresses the original code into memory before transferring execution to the real entry point.

From an analysis perspective, packed binaries present a significant challenge because the majority of the malicious code is hidden within the compressed payload and is not visible in the file on disk. High entropy values in the binary sections are a reliable indicator of packing, as compressed or encrypted data naturally exhibits near maximum entropy. In the case of the sample analyzed in this report, an entropy value of 7.875 was recorded for the packed binary, confirming the presence of a packer. After unpacking using UPX, the binary expanded from 445,231 bytes to 792,879 bytes, revealing the full extent of its imported functions and embedded strings.

2.4 LOCKSCREEN RANSOMWARE

LockScreen ransomware represents a category of malicious software that prevents victims from accessing their desktop or files by displaying a fullscreen window that cannot be dismissed through normal means. Unlike encryption based ransomware, which encrypts the victim's files and demands payment for a decryption key, LockScreen ransomware typically focuses on denying access to the system interface itself. This approach is generally simpler to implement but can be equally effective in causing disruption, particularly against less technically experienced users.

LockScreen malware commonly achieves its effect through a combination of Windows API calls that create a fullscreen topmost window, disable user input, and suppress normal system interactions such as the taskbar and desktop. Research into LockScreen samples has identified several common techniques including the use of SetWindowPos to place the lockscreen window above all others, BlockInput to prevent keyboard and mouse interaction, and ShowWindow to control the visibility of other windows. The sample analyzed in this report exhibits all of these characteristics, as confirmed through both static string analysis and dynamic debugging observations.

2.5 INDICATORS OF COMPROMISE AND THREAT INTELLIGENCE

Indicators of compromise are pieces of forensic evidence that suggest a system may have been compromised by malicious activity. They are a fundamental component of threat intelligence, enabling security teams to detect, respond to, and attribute attacks. Common indicators of compromise extracted from malware samples include hardcoded IP addresses, domain names, file paths, registry keys, mutexes, and suspicious API call patterns.

Reverse engineering plays a central role in the extraction of indicators of compromise, as many of the most valuable artifacts are embedded within the binary and cannot be observed through behavioral monitoring alone. The combination of static string analysis using tools such as PEStudio and dynamic debugging using tools such as x32dbg has been shown to be an effective methodology for extracting a comprehensive set of indicators from complex malware samples. The findings presented in this report demonstrate the practical application of this methodology to a real world malware sample.

CHAPTER 3

BINARY STRUCTURE ANALYSIS, DECOMPILATION AND DEBUGGING

3.1 BINARY STRUCTURE ANALYSIS

The first phase of the reverse engineering process involved a thorough examination of the binary structure of the malware sample using PEStudio. This static analysis phase was conducted on the original packed binary before any unpacking was performed, allowing for the identification of the packing mechanism and the confirmation of key file properties.

3.1.1 FILE PROPERTIES AND PE HEADER

Upon loading ransomware.exe into PEStudio, the following file properties were identified. The binary is a 32-bit portable executable with a graphical user interface subsystem, compiled using Microsoft Visual Studio 2010 and linked with Microsoft Linker version 10.0. The compilation timestamp recorded within the PE header is Sunday, January 29, 2012, at 21:32:28 UTC, suggesting the malware was originally developed over a decade ago and may have been redistributed or repackaged more recently. The file size of the packed binary is 445,231 bytes, with an entropy value of 7.875, which is extremely close to the theoretical maximum of 8.0 and is a strong indicator of compression or encryption. The version information embedded within the binary identifies it as a CompiledScript compiled using AutoIt version 3, confirming the use of the AutoIt scripting platform as the malware development environment.

The entry point of the packed binary is located at memory address 0x000CEE90, within the section labeled idata. The presence of UPX section markers UPX0 and UPX1 in the strings output, along with the UPX version string 3.07, confirmed that the binary was packed using UPX version 3.07. The binary also exhibited a self-modifying code characteristic flagged by PEStudio, which is consistent with the UPX unpacking stub modifying memory at runtime to restore the original executable code before transferring control to the real entry point.

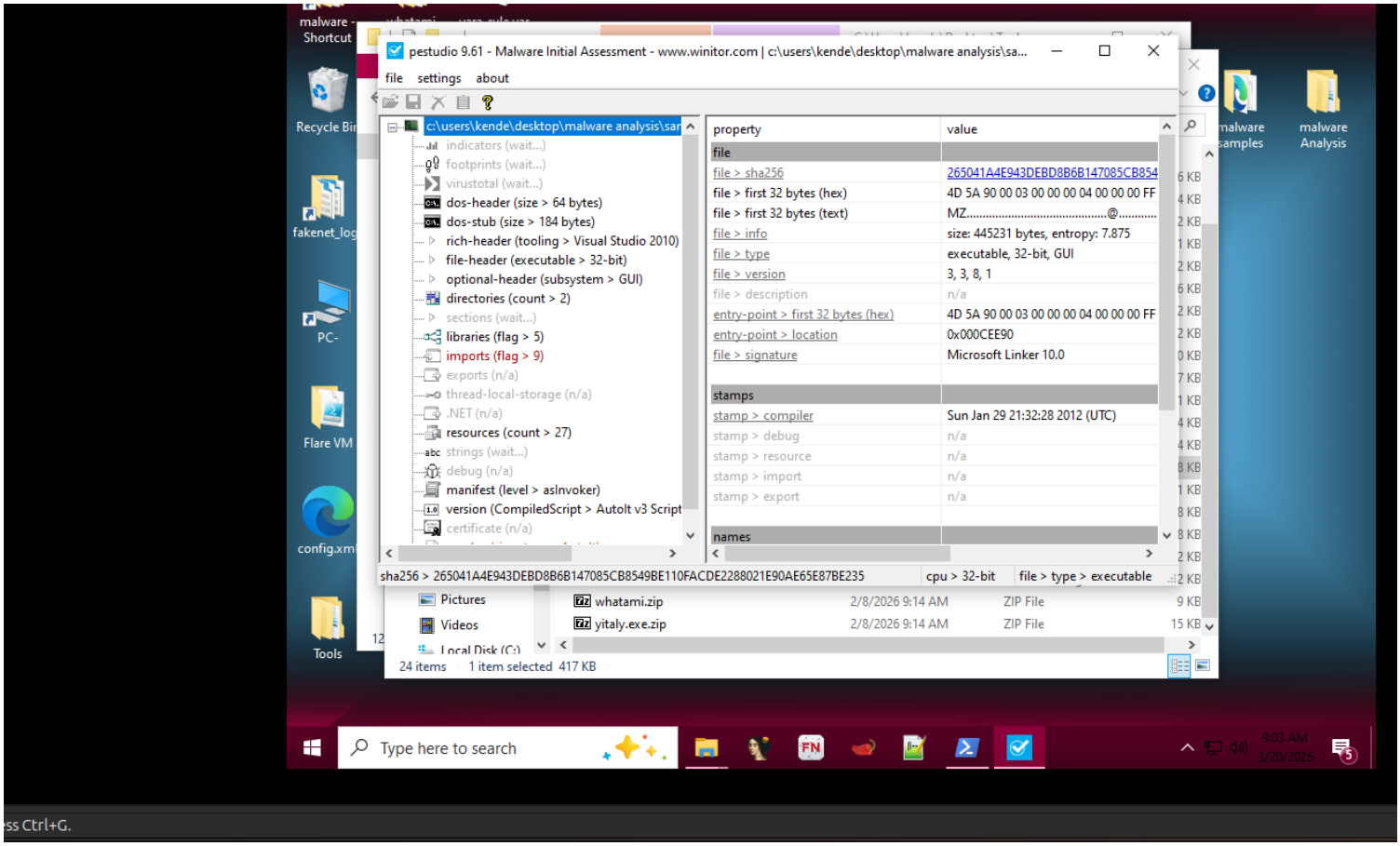

Figure 3.1 PEStudio file properties and PE header of the packed binary

3.1.2 IMPORTED LIBRARIES AND SUSPICIOUS API CALLS

The packed binary contained only 21 visible imports, of which 9 were flagged as suspicious by PEStudio. This relatively small import table is typical of packed binaries, where the majority of the actual API calls are hidden within the compressed payload and are only resolved at runtime after unpacking. Among the flagged imports in the packed binary were VirtualProtect and VirtualAlloc, which are commonly used in unpacking routines to allocate and modify memory permissions, as well as EnumProcesses for process enumeration, FtpOpenFileW for FTP connectivity, and WNetGetConnectionW for network drive access. The presence of these APIs in the packed binary alone was sufficient to confirm malicious intent even before the full unpacking was performed.

After unpacking the binary using UPX, the import table expanded dramatically from 21 to

513 total imports, with 156 flagged as suspicious. This significant increase confirmed that the packer had been concealing the majority of the malware's functional capabilities. The unpacked binary revealed a comprehensive set of API calls spanning process injection, file manipulation, network communication, registry access, and system information harvesting. Key flagged imports identified in the unpacked binary included OpenProcess, VirtualAllocEx, WriteProcessMemory, and ReadProcessMemory, which together form a classic process injection sequence. Additional network related imports included InternetOpenW, InternetConnectW, HttpOpenRequestW, HttpSendRequestW, InternetReadFile, FtpOpenFileW, and a full set of raw socket functions including setsockopt, sendto, recvfrom, and select, indicating the malware is capable of communicating over both HTTP and FTP protocols as well as through raw network sockets.

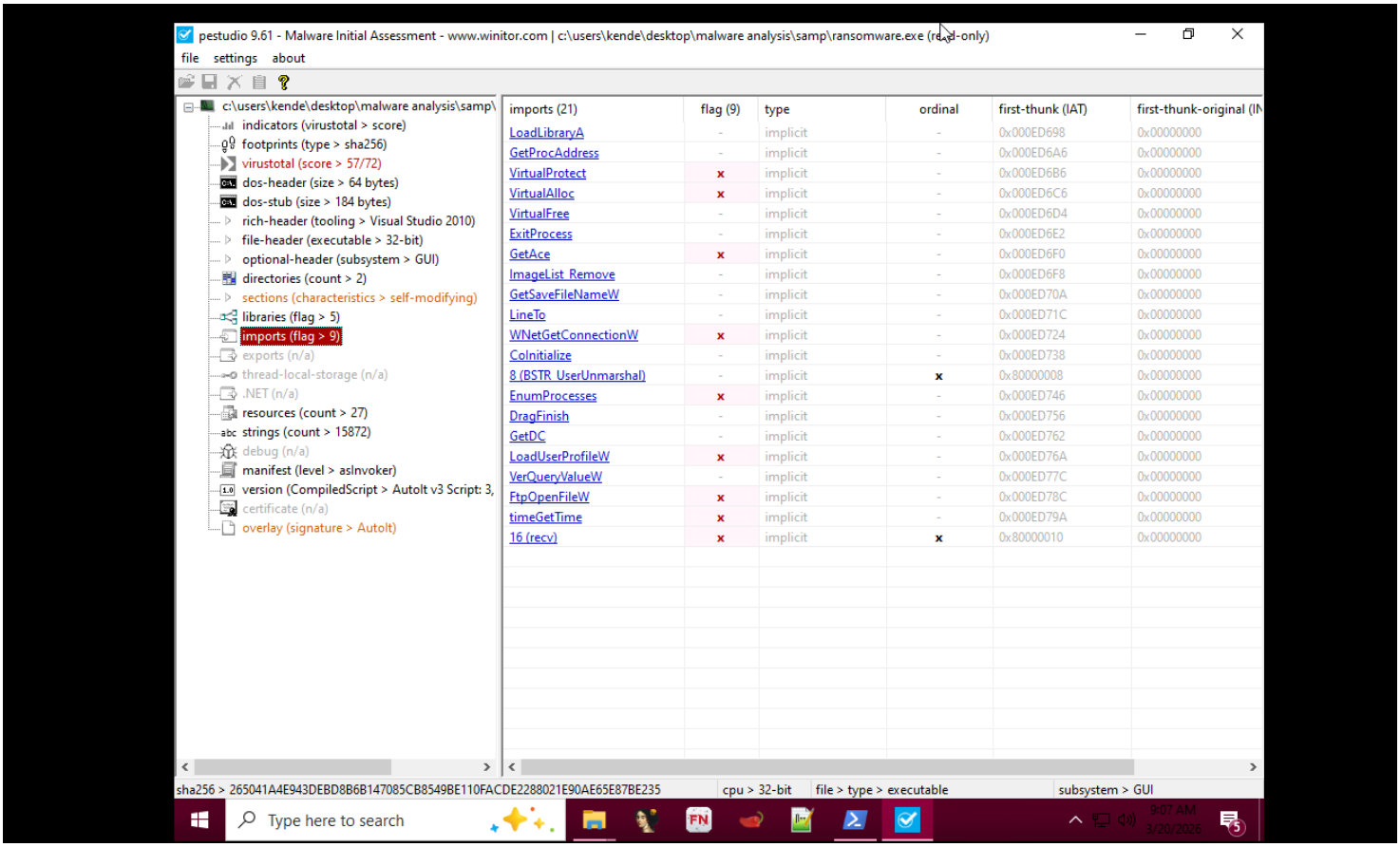

Figure 3.2 Flagged imports in the packed binary

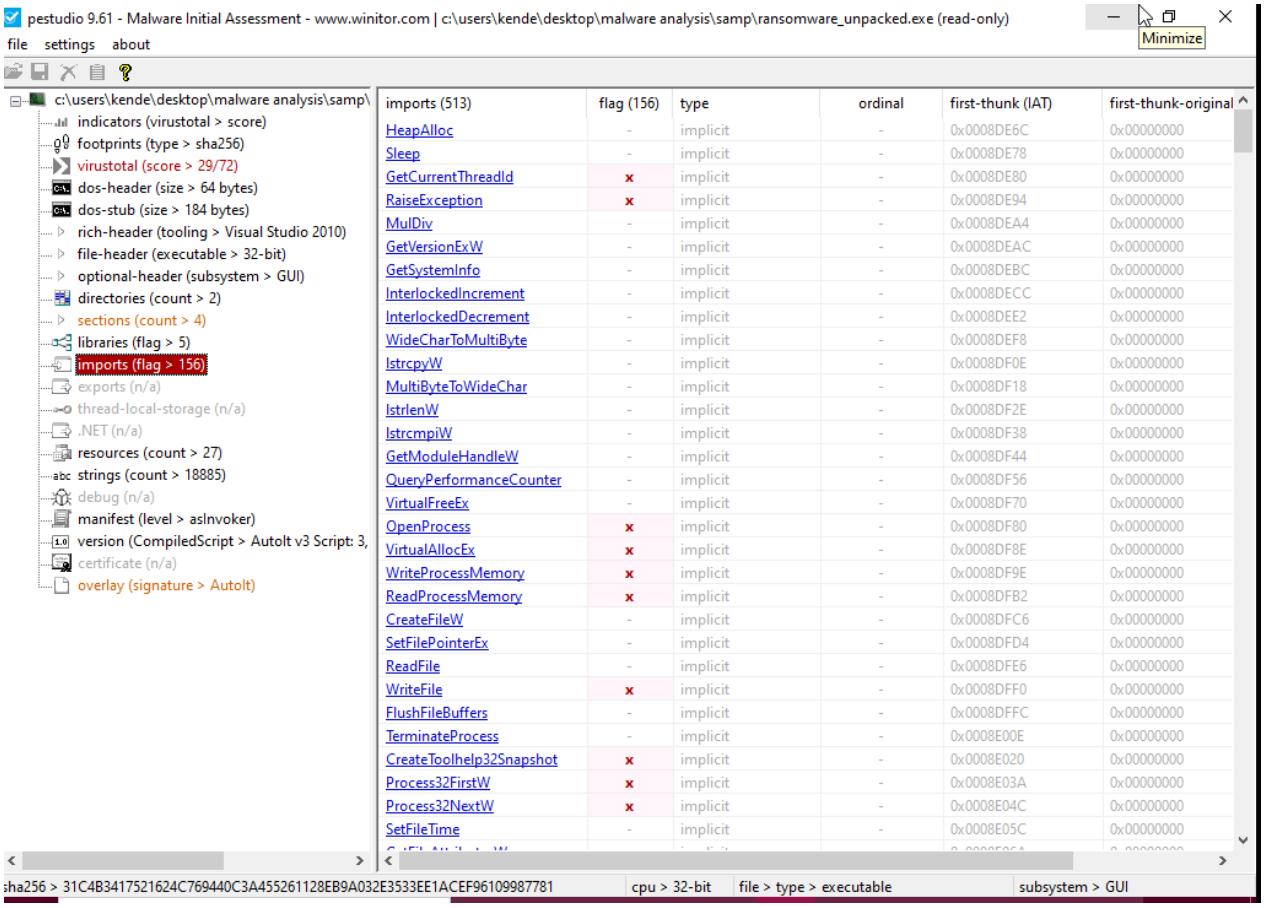

Figure 3.3 Flagged imports in the unpacked binary showing expanded import table

3.1.3 UPX UNPACKING

To facilitate deeper static and dynamic analysis, the binary was unpacked using the UPX command line tool. The unpacking command was executed within the virtual machine using Windows PowerShell, producing a decompressed output file named ransomware_unpacked.exe. The unpacking process reported a compression ratio of 56.15 percent, with the file expanding from 445,231 bytes to 792,879 bytes, nearly doubling in size. This significant expansion confirmed that the UPX packer had been compressing a substantial portion of the malware's code and data. Following unpacking, a new SHA256 hash was computed for the unpacked binary, yielding the value 31C4B3417521624C769440C3A455261128EB9A032E3533EE1ACEF96109987781, which was recorded for reference alongside the original packed binary hash.

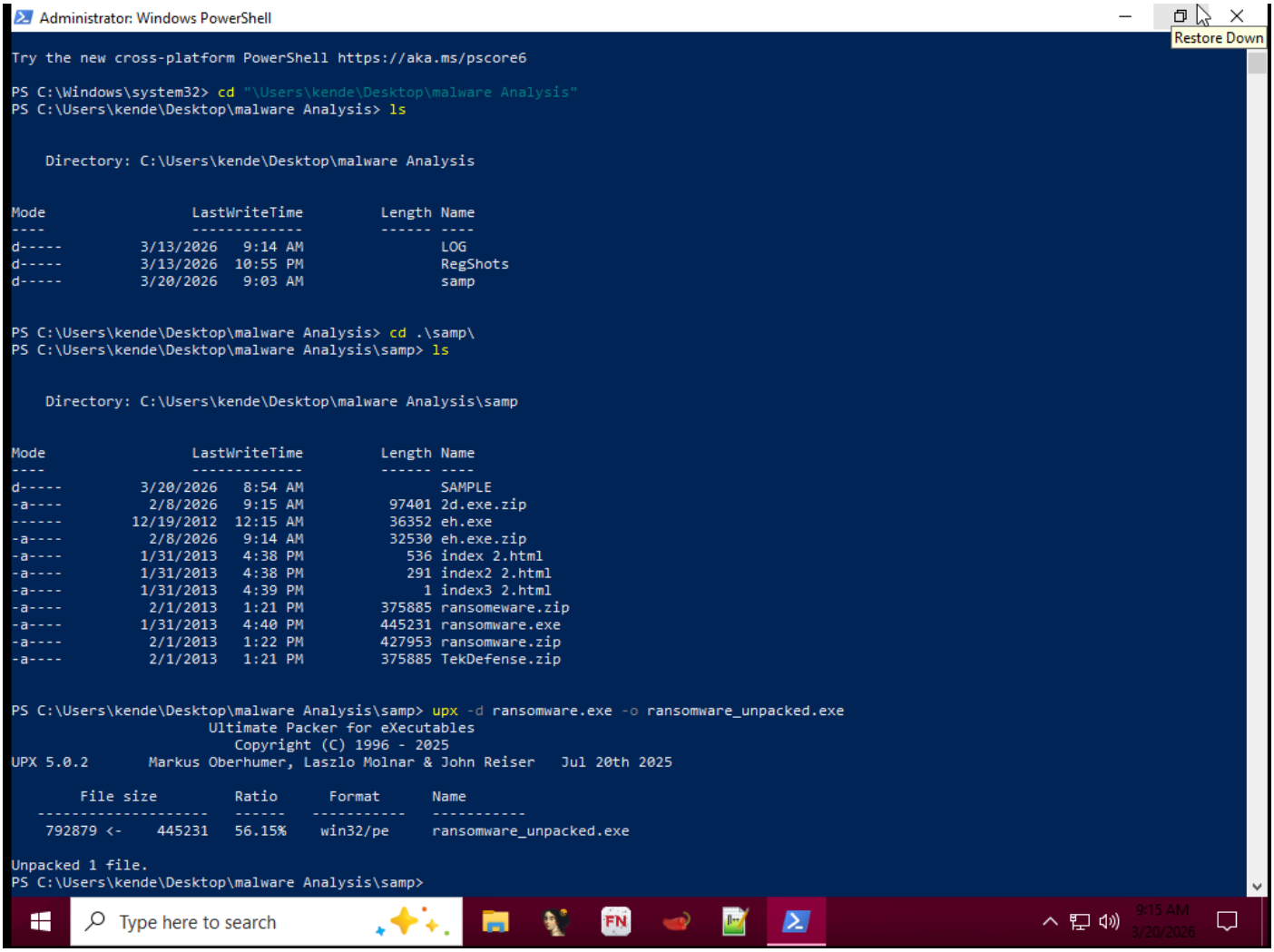

Figure 3.4 UPX unpacking output showing compression ratio and file size expansion

3.1.4 STRING ANALYSIS

String extraction from the unpacked binary yielded 18,885 strings, a significant increase from the 15,872 strings visible in the packed binary. Sorting the strings by the PEStudio flag value revealed a number of highly significant artifacts. The AutoIt compiler watermark string confirmed the AutoIt origin of the binary. The presence of ICMP related strings including IcmpCreateFile, IcmpCloseHandle, and IcmpSendEcho confirmed that the malware is capable of performing network ping operations, likely for host discovery or connectivity checking. System reconnaissance strings including USERNAME, IPADDRESS1 through IPADDRESS4, OSVERSION, CPUARCH, PROCESSORARCH, OSLANG, OSSERVICEPACK, OSBUILD, KBLAYOUT, and USERDNSDOMAIN confirmed that the malware performs comprehensive fingerprinting of the victim system before deploying its payload.

Additional strings of significance included SeDebugPrivilege and SeShutdownPrivilege, indicating that the malware requests elevated system privileges, and SW_LOCK, which directly references the screen locking functionality. Window control strings such as WINMINIMIZEALL, WINSETONTOPP, WINSETTRANS, WINWAITNOTACTIVE, and WINWAITCLOSE confirmed the mechanism by which the malware implements its lockscreen behavior, minimizing all existing windows and placing its own fullscreen window on top with controlled transparency. Registry related strings including references to HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT, HKEY_CURRENT_CONFIG, and HKEY_USERS confirmed full registry access capability across all major registry hives.

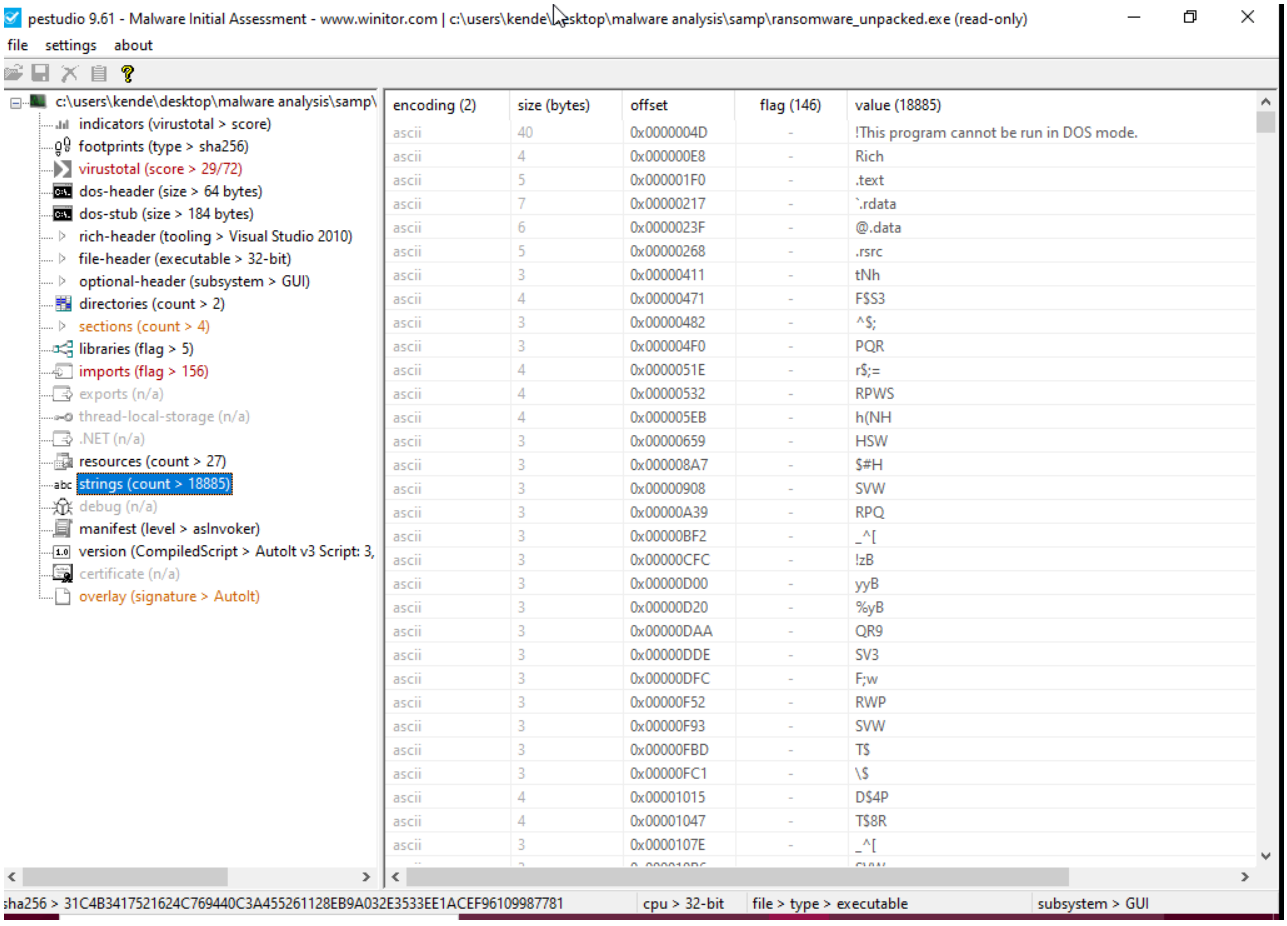

Figure 3.5 Flagged strings extracted from the unpacked binary in PEStudio

3.2 CODE DECOMPILATION

The unpacked binary was loaded into Ghidra for decompilation and code level analysis. A new project named RansomwareAnalysis was created and the binary was imported using the default analysis settings. Ghidra identified a total of 2,222 functions within the unpacked binary, which is consistent with the large AutoIt runtime engine embedded within the executable.

3.2.1 ENTRY POINT AND MAIN FUNCTION IDENTIFICATION

The entry point of the binary as identified by Ghidra is located at address 0x004165C1 within the .text section, which is consistent with the entry point location reported by PEStudio for the unpacked binary. The entry function contained a minimal set of instructions, calling __security_init_cookie for stack protection initialization followed by __tmainCRTStartup, which is the standard Microsoft Visual C++ runtime startup routine. Examination of the __tmainCRTStartup function revealed references to ADVAPI32.DLL imports including CreateProcessWithLogonW and LogonUserW, indicating that the malware startup routine involves the creation of processes under alternative user credentials, which is a known privilege escalation technique.

To identify the primary malicious function, the Functions window in Ghidra was sorted by function size in descending order. The largest function in the binary, identified as FUN_00437262, had a size of 30,796 bytes, significantly larger than any other function in the binary. This function was renamed to main_malicious_logic to reflect its identified role as the primary execution function of the malware. The function accepts ten parameters and contains thousands of lines of decompiled code, consistent with the AutoIt runtime interpreter processing a compiled script.

3.2.2 IDENTIFIED FUNCTIONS AND CAPABILITIES

Beyond the primary malicious function, several other notable functions were identified during the Ghidra analysis. The second largest function, FUN_00404170 with a size of 19,272 bytes, and the third largest, FUN_0044cf17 with a size of 12,805 bytes, are likely responsible for supporting operations such as string processing, network communication handling, and GUI management. The presence of CreateProcessWithLogonW and LogonUserW within

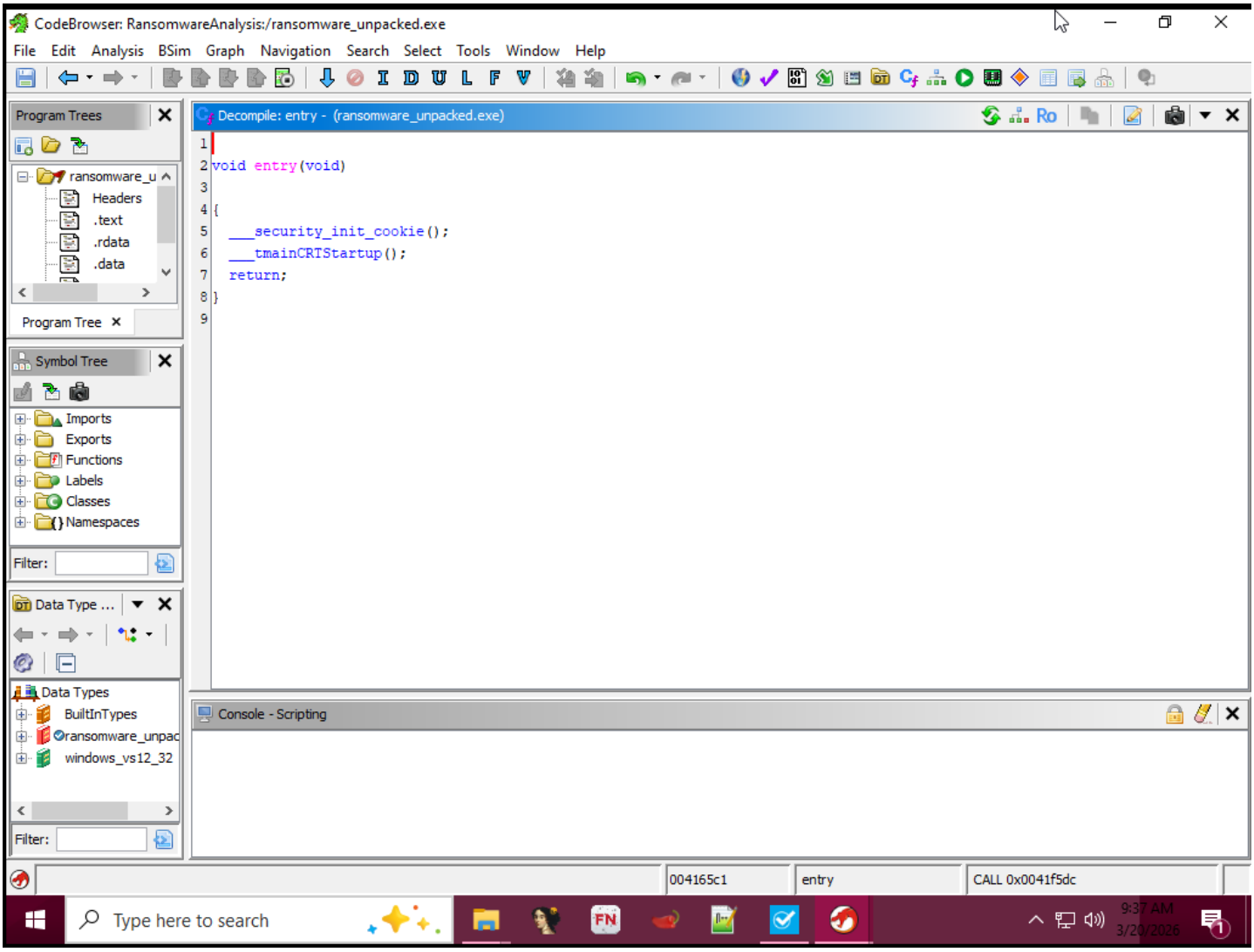

Figure 3.6 Ghidra decompiler view showing the entry point function

the import resolution chain confirmed that the malware includes functionality for executing processes under different user credentials, which could be used for lateral movement or privilege escalation within a compromised environment.

The decompilation of the main function revealed extensive use of pointer arithmetic and byte level operations, consistent with the AutoIt runtime processing an embedded compiled script. The recursive nature of certain function calls within the main function suggests the presence of an interpreter loop, which is characteristic of AutoIt compiled executables where the embedded runtime engine iterates through script opcodes to execute the malicious payload.

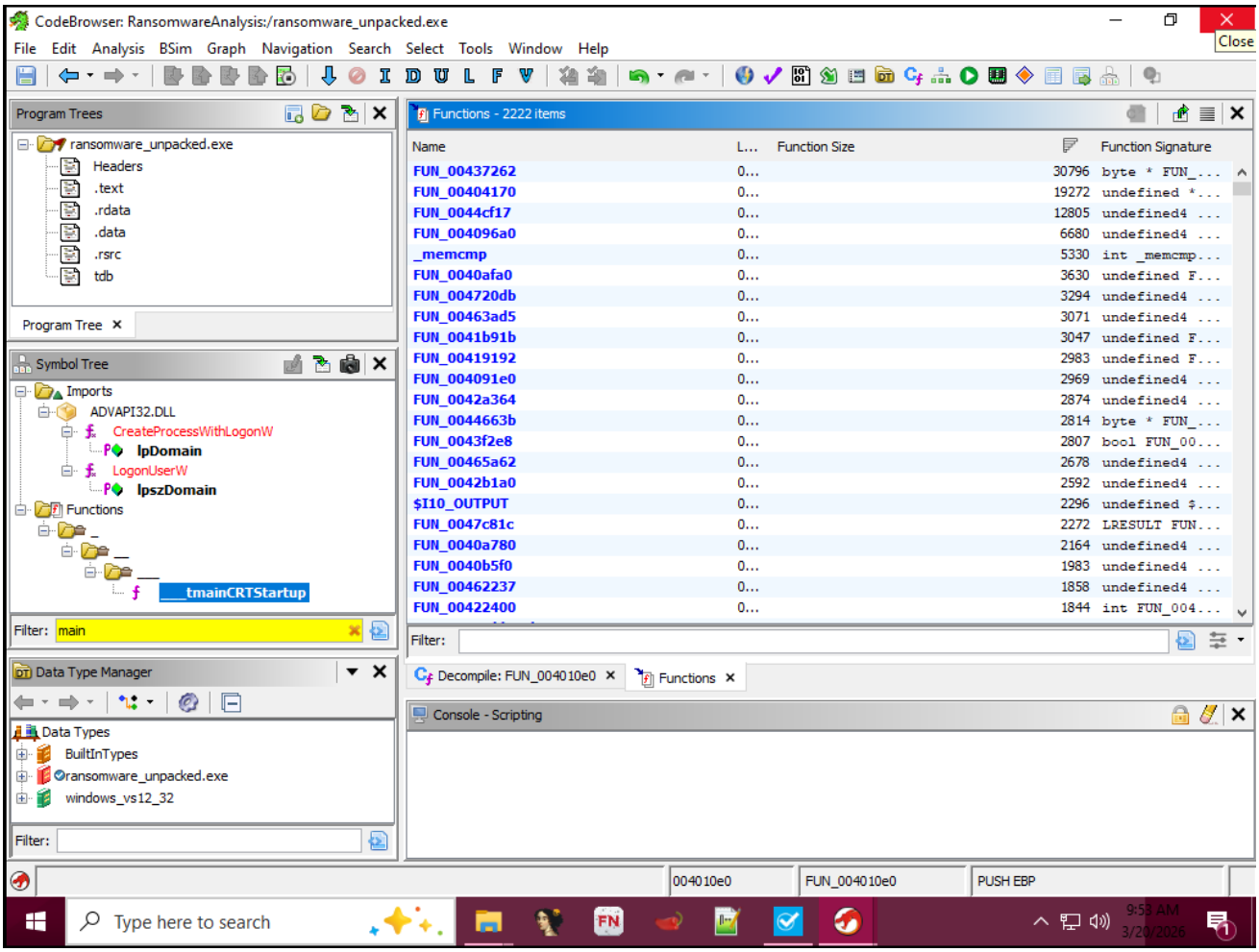

Figure 3.7 Ghidra functions list sorted by size showing the largest functions

3.3 DEBUGGING AND RUNTIME INSPECTION

Dynamic analysis of the malware was conducted using x32dbg, the 32-bit variant of the x64dbg debugger. The original packed binary was loaded into x32dbg for debugging, as analyzing the runtime behavior of the packed binary provides a more accurate representation of how the malware would behave on a victim system.

3.3.1 ENTRY POINT AND INITIAL EXECUTION

Upon loading the binary, x32dbg paused execution at the system breakpoint within ntdll.dll, which is the standard behavior for the debugger before transferring control to the target process. After pressing F9 to continue execution, the debugger paused again at the malware entry point at address 0x004CEE90, confirmed by the status bar message indicating an INT3 breakpoint at ransomware.OptionalHeader.AddressOfEntryPoint. The first instruction at the entry point was a pushad instruction, which saves all general purpose registers to the stack. This is the characteristic first instruction of a UPX unpacking stub, confirming that the packed binary begins execution by saving the register state before proceeding with the decompression of the original code.

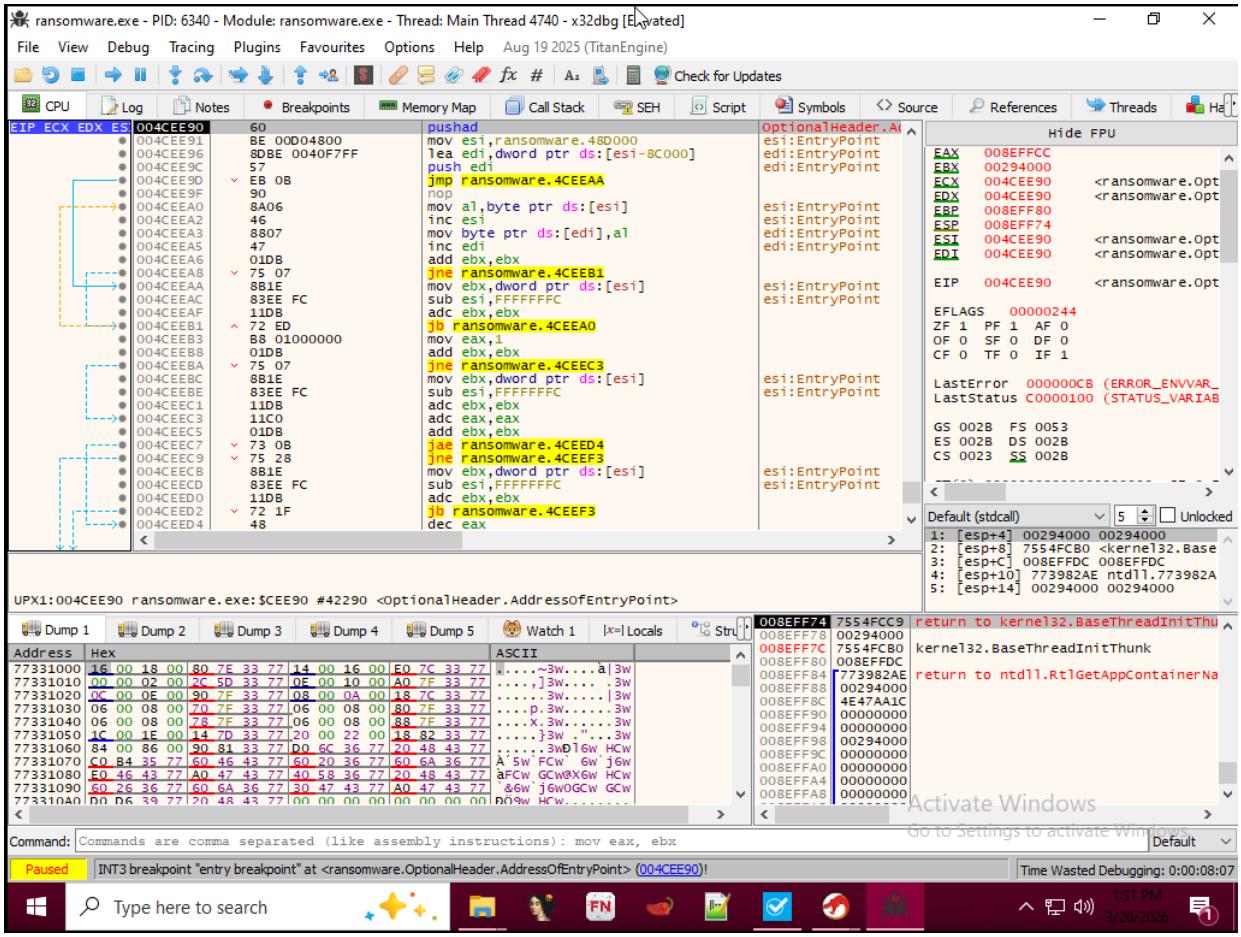

Figure 3.9 x32dbg paused at the malware entry point showing the UPX unpacking stub

3.3.2 DYNAMIC LIBRARY LOADING

Examination of the x32dbg log tab revealed a comprehensive list of dynamic link libraries loaded by the malware at runtime. The libraries loaded included shell32.dll for shell operations and file management, ole32.dll and oleaut32.dll for COM object handling and script automation, psapi.dll for process enumeration, mpr.dll for network provider access, userenv.dll for user profile loading, wininet.dll for internet connectivity, winmm.dll for multimedia and timing functions, wsock32.dll and ws2_32.dll for Windows socket operations, and imm32.dll for input method and keyboard handling. The loading of this extensive set of libraries at runtime is consistent with the AutoIt runtime engine initializing its full set of supported functions before executing the embedded malicious script.

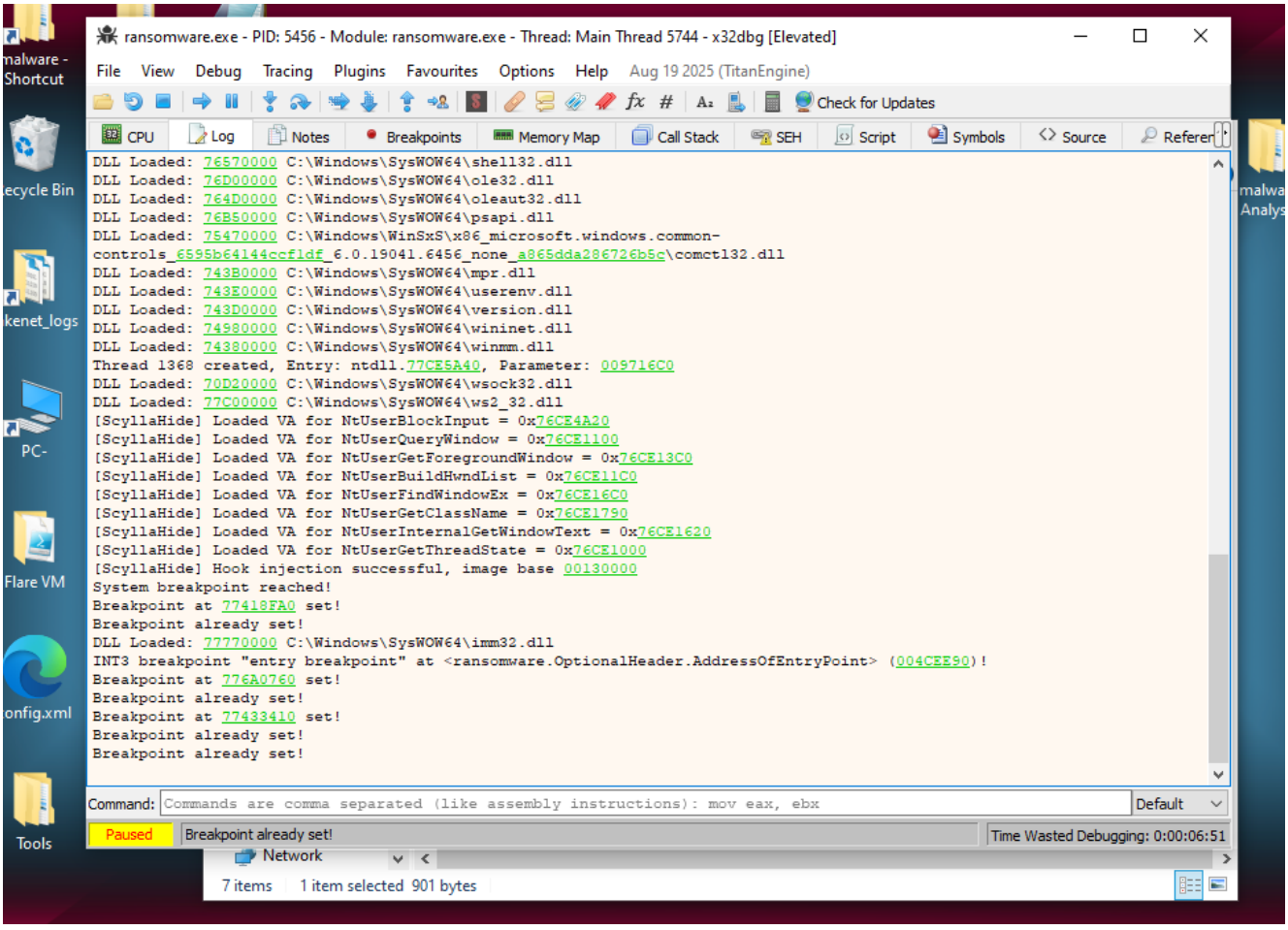

Figure 3.10 x32dbg log tab showing runtime library loading and ScyllaHide hooks

3.3.3 ANTI-DEBUGGING BEHAVIOR

The x32dbg log also revealed that the ScyllaHide anti-detection plugin successfully hooked several user input and window management functions within the malware process, including NtUserBlockInput, NtUserQueryWindow, NtUserGetForegroundWindow, NtUserBuildHwndList, NtUserFindWindowEx, NtUserGetClassName, NtUserInternalGetWindowText, and NtUserGetThreadState. The targeting of these specific functions confirms that the malware actively monitors and controls the windowing environment and blocks user input as part of its lockscreen deployment mechanism. The fact that all three breakpoints set on CreateWindowExA, ShowWindow, and VirtualProtect recorded zero hits despite the lockscreen payload executing successfully indicates that the malware employs evasion techniques to bypass standard API level monitoring, likely through direct system calls or alternative code paths that circumvent the hooked API entry points.

3.3.4 LOCKSCREEN PAYLOAD EXECUTION

During the debugging session, the malware successfully deployed its lockscreen payload, resulting in the complete takeover of the virtual machine desktop. The screen turned entirely white and all normal desktop interaction was disabled, consistent with the behavior of a LockScreen ransomware sample. This observation confirmed the primary malicious capability of the sample and validated the findings obtained through static analysis. The virtual machine was subsequently restored to a clean snapshot to prevent any further impact on the analysis environment.

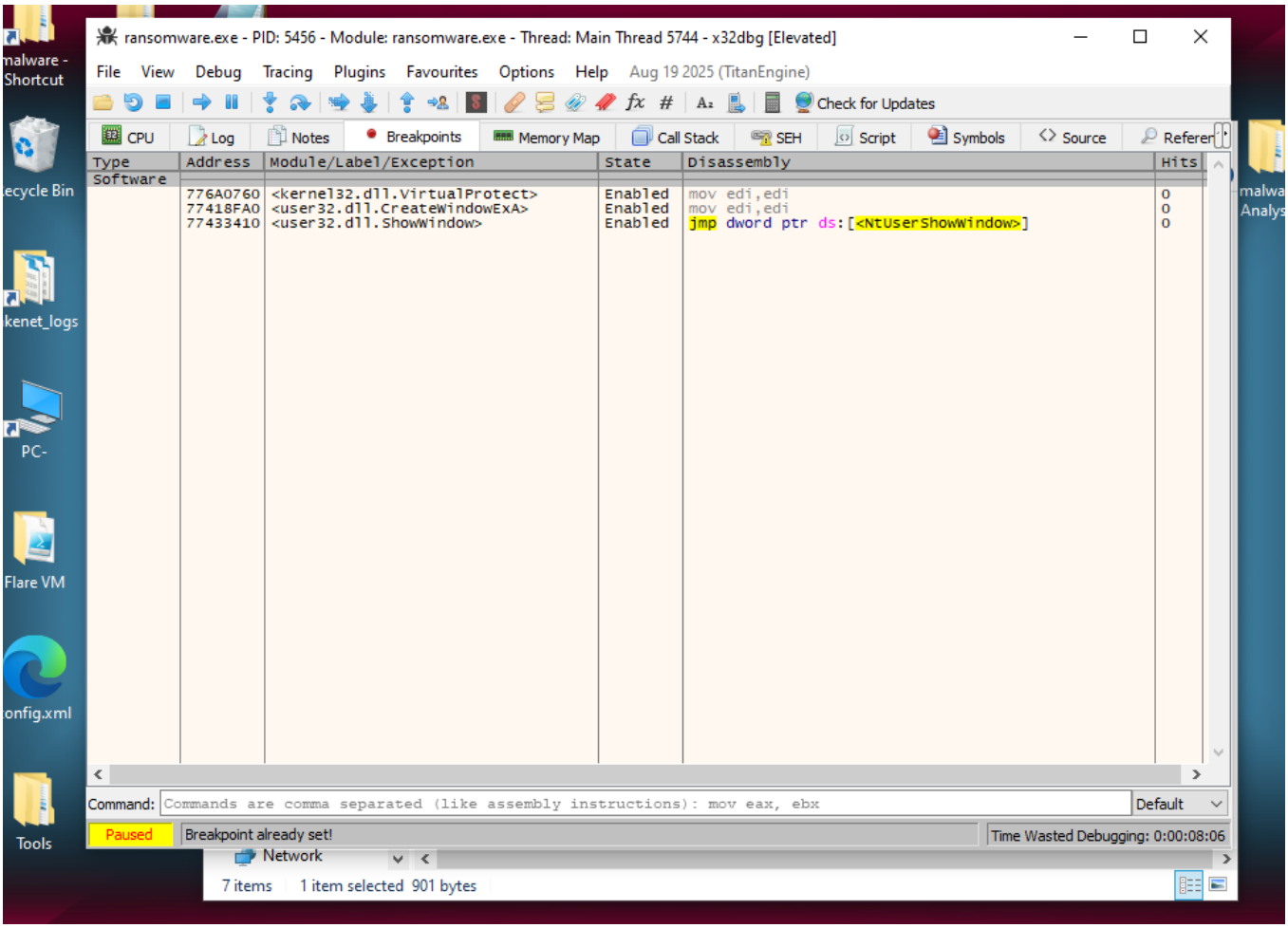

Figure 3.11 x32dbg breakpoints tab showing zero hits on monitored API calls

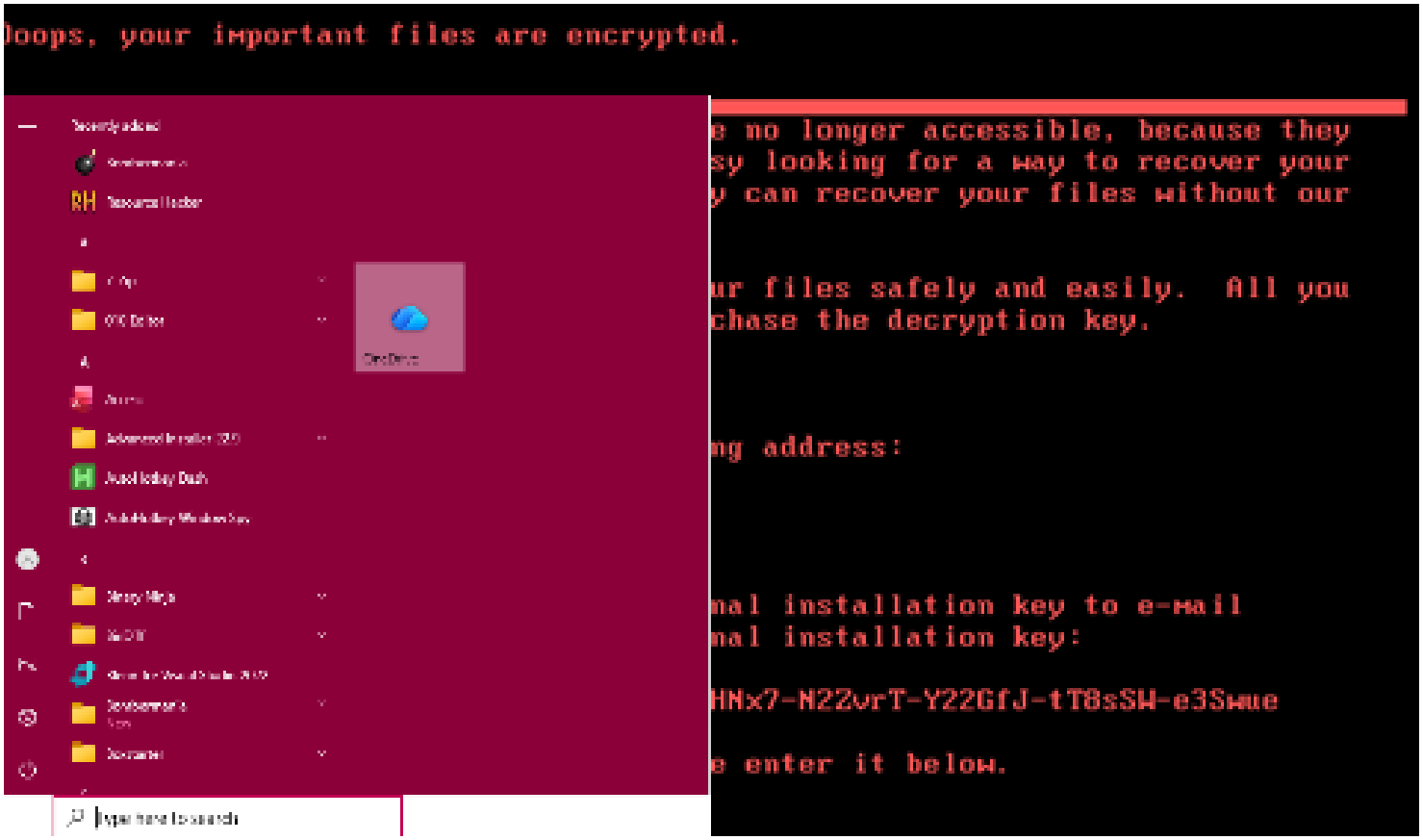

Figure 3.12 Lockscreen payload deployed by the malware during debugging session

CHAPTER 4

MALWARE CAPABILITIES AND INDICATORS OF COMPROMISE

4.1 MALWARE CAPABILITY ANALYSIS

The reverse engineering process conducted across Chapters 3 revealed a comprehensive set of malicious capabilities embedded within the ransomware.exe sample. This chapter consolidates those findings into a structured capability analysis and presents the full set of indicators of compromise extracted during the investigation. The capabilities identified span system reconnaissance, privilege escalation, process injection, network communication, input manipulation, and screen locking, confirming that this sample is a fully featured malicious tool despite its relatively simple outward behavior.

4.1.1 SYSTEM RECONNAISSANCE

One of the most significant capabilities identified through string analysis was the malware's ability to perform comprehensive fingerprinting of the victim system prior to deploying its payload. The presence of AutoIt environment variables including USERNAME, USERDOMAIN, USERDNSDOMAIN, USERPROFILE, IPADDRESS1, IPADDRESS2, IPADDRESS3, and IPADDRESS4 confirmed that the malware collects the logged in username, domain information, DNS domain name, user profile path, and all network adapter IP addresses from the victim machine. Additionally the variables OSVERSION, OSBUILD, OSTYPE, OSLANG, OSSERVICEPACK, CPUARCH, PROCESSORARCH, KBLAYOUT, DESKTOPWIDTH, DESKTOPHEIGHT, DESKTOPDEPTH, and DESKTOPREFRESH confirmed that the malware harvests detailed information about the operating system version and build, processor architecture, keyboard layout, and screen resolution. This level of system profiling is consistent with malware that reports victim information to a remote operator or uses the collected data to tailor its behavior to the specific target environment.

4.1.2 PRIVILEGE ESCALATION

The malware was found to request two significant Windows privilege tokens during execution. The first, SeDebugPrivilege, grants the process the ability to open and manipulate other

processes regardless of their security descriptors, which is a prerequisite for process injection techniques. The second, SeShutdownPrivilege, grants the process the ability to force a system shutdown, which could be used to prevent recovery or to trigger a reboot into a compromised state. The presence of both privilege tokens, combined with the identified process injection API calls OpenProcess, VirtualAllocEx, WriteProcessMemory, and ReadProcessMemory, confirms that the malware is capable of injecting code into running processes with elevated permissions.

4.1.3 PROCESS INJECTION

The combination of OpenProcess, VirtualAllocEx, WriteProcessMemory, and ReadProcessMemory identified in the unpacked binary's import table represents a textbook process injection sequence. This technique involves opening a handle to a target process, allocating memory within it, writing malicious code into the allocated region, and then executing the injected code within the context of the target process. Process injection is commonly used by malware to hide its activity within legitimate processes, evade detection by security tools, and maintain persistence within a running system. The additional presence of CreateProcessWithLogonW and LogonUserW in the import resolution chain confirmed that the malware can also create entirely new processes under different user credentials, further extending its privilege escalation and lateral movement capabilities.

4.1.4 NETWORK COMMUNICATION

The network communication capabilities of the malware were among the most extensive identified during the analysis. The unpacked binary contained a full suite of WinINet API calls including InternetOpenW for initializing internet connections, InternetConnectW for establishing connections to remote servers, HttpOpenRequestW and HttpSendRequestW for sending HTTP requests, InternetReadFile for downloading content, InternetCrackUrlW for parsing URLs, FtpOpenFileW and FtpGetFileSize for FTP file operations, and InternetOpenUrlW for directly opening URLs. In addition to these high level network functions, the malware also imported a full set of raw socket functions including setsockopt, sendto, recvfrom, select, htons, and ntohs, enabling it to communicate at a lower level through raw network sockets. The presence of IcmpCreateFile, IcmpSendEcho, and IcmpCloseHandle further confirmed the ability to perform ICMP ping operations, likely for host discovery

or network connectivity checking. The broadcast IP address 255.255.255.255 identified in the string analysis suggests that the malware may also perform network-wide broadcast communications.

4.1.5 INPUT MANIPULATION AND SCREEN LOCKING

The primary payload of the malware, as confirmed through both static analysis and dynamic debugging, is a screen locking mechanism that prevents the victim from accessing the desktop. The string analysis revealed a comprehensive set of AutoIt window control commands that implement this behavior, including WINMINIMIZEALL to minimize all open windows, WINSETONTOPP to place the lockscreen window above all others, WINSETTRANS to control window transparency, WINWAITNOTACTIVE and WINWAITCLOSE to monitor the state of other windows, and SW_LOCK to directly invoke the screen lock. The ScyllaHide hooks observed during debugging confirmed that the malware targets NtUserBlockInput to disable keyboard and mouse interaction, preventing the victim from dismissing the lockscreen window through normal means. The SwapMouseButtons string further suggested that the malware may also manipulate mouse button assignments to confuse the victim and complicate any attempt to interact with the locked screen.

4.1.6 REGISTRY MANIPULATION

The malware demonstrated full registry access capability across all five major Windows registry hives, HKEY_LOCAL_MACHINE, HKEY_CURRENT_USER, HKEY_CLASSES_-ROOT, HKEY_CURRENT_CONFIG, and HKEY_USERS. The presence of registry value type strings including REG_SZ, REG_EXPAND_SZ, REG_MULTI_SZ, REG_DWORD, REG_QWORD, and REG_BINARY confirmed that the malware is capable of reading and writing all standard registry value types. The SOFTWARE\Classes\and \CLSID registry paths identified in the string analysis suggested that the malware may modify COM object registrations, which is a known persistence technique that allows malware to survive system reboots by hijacking legitimate COM object references.

4.2 INDICATORS OF COMPROMISE

The following section presents a consolidated list of all indicators of compromise extracted during the reverse engineering analysis. These indicators can be used by security teams

to detect the presence of this malware on compromised systems and to develop detection signatures for intrusion detection systems and endpoint security tools.

4.2.1 FILE INDICATORS

The primary file indicators associated with this malware sample are presented in the table below. These hash values can be used to identify the presence of either the packed or unpacked variant of the binary on a compromised system.

Table 4.1 File Indicators of Compromise

Indicator	**Value**
File Name	ransomware.exe
SHA256 (Packed)	265041a4e943debd8b6b147085cb8549be110facde 2288021e90ae65e87be235
MD5 (Packed)	63bab74409c514ed8548b1f33d0acedc
SHA1 (Packed)	abc6bb8dd01fa83d7fd92182601b868d2b6dd1ea
SHA256 (Unpacked)	31C4B3417521624C769440C3A455261128EB9A032E 3533EE1ACEF96109987781
File Size (Packed)	445,231 bytes
File Size (Unpacked)	792,879 bytes
Packer	UPX version 3.07
Compiler	AutoIt v3 Script, Microsoft Linker 10.0

4.2.2 NETWORK INDICATORS

The network indicators identified during the analysis are presented below. These can be used to detect outbound network communications from compromised systems.

Table 4.2 Network Indicators of Compromise

Indicator Type	**Value**
Broadcast IP Address	255.255.255.255
Wildcard Bind Address	0.0.0.0
Network Protocols	HTTP, FTP, ICMP, Raw Sockets
Network Libraries	wininet.dll, wsock32.dll, ws2_32.dll, icmp.dll

4.2.3 REGISTRY INDICATORS

The registry indicators identified during the analysis are presented below. Security teams can monitor these registry locations for unauthorized modifications as part of a detection strategy.

Table 4.3 Registry Indicators of Compromise

Registry Hive	**Significance**
HKEY_LOCAL_MACHINE	Core system configuration access
HKEY_CURRENT_USER	Current user settings modification
HKEY_CLASSES_ROOT	File association and COM object tampering
HKEY_CURRENT_CONFIG	Hardware configuration access
HKEY_USERS	All user profiles access
SOFTWARE\Classes\	COM object registration modification
SYSTEM\CurrentControlSet\Control\Nls\Language	System language settings access
Control Panel\Appearance	Desktop appearance modification
Control Panel\Mouse	Mouse settings manipulation

4.2.4 BEHAVIORAL INDICATORS

The behavioral indicators identified during both static and dynamic analysis are presented below. These indicators describe observable behaviors that can be used to identify the presence of this malware on a running system.

Table 4.4 Behavioral Indicators of Compromise

Behavior	**Description**
Screen Locking	Deploys a fullscreen white window that disables all desktop interaction
Input Blocking	Disables keyboard and mouse input via NtUserBlockInput
Process Injection	Injects code into running processes using OpenProcess and WriteProcessMemory
Privilege Escalation	Requests SeDebugPrivilege and SeShutdownPrivilege
System Fingerprinting	Collects username, IP addresses, OS version, CPU architecture, and keyboard layout
Network Reconnaissance	Performs ICMP ping operations via IcmpSendEcho
Registry Modification	Accesses and modifies all major registry hives
Mouse Manipulation	Swaps mouse buttons via SwapMouseButtons
Anti-Debugging	Bypasses API level breakpoints through direct system calls

4.2.5 PRIVILEGE AND SECURITY INDICATORS

The privilege related indicators identified during the analysis are presented below. The presence of either of these privilege tokens in a process token on a monitored system should be treated as a strong indicator of malicious activity.

Table 4.5 Privilege Indicators of Compromise

Privilege Token	Significance
SeDebugPrivilege	Enables process injection and memory manipulation across all running processes
SeShutdownPrivilege	Enables forced system shutdown

4.3 COMPARISON WITH DYNAMIC ANALYSIS FINDINGS

The findings of this reverse engineering analysis are broadly consistent with the behavioral observations recorded during the dynamic analysis assignment conducted on the same sample. The lockscreen payload observed during dynamic analysis is fully explained by the static string evidence identifying WINMINIMIZEALL, WINSETONTOPP, SW_LOCK, and NtUserBlockInput as core components of the malware's window management and input blocking routines. The network activity observed during dynamic analysis is consistent with the comprehensive suite of WinINet, FTP, and raw socket APIs identified in the unpacked binary's import table. The system information collection observed behaviorally is directly supported by the AutoIt environment variable strings extracted during static analysis, which enumerate every category of system information that the malware is capable of harvesting.

The reverse engineering analysis adds significant depth to the dynamic analysis findings by revealing the internal mechanisms behind the observed behaviors. Where dynamic analysis could only confirm that the lockscreen was deployed, the reverse engineering analysis identified the specific API calls, AutoIt commands, and window management techniques used to implement it. Where dynamic analysis observed network connections, the reverse engineering analysis revealed the full protocol stack available to the malware including HTTP, FTP, ICMP, and raw sockets. This complementary relationship between dynamic and static analysis demonstrates the value of combining both approaches when investigating complex malware samples, as neither method alone would have provided a complete picture of the malware's capabilities and design.

4.4 CONCLUSION

This report has presented a comprehensive reverse engineering analysis of ransomware.exe, a LockScreen ransomware sample compiled using the AutoIt version 3 scripting language and protected with UPX packing. Through a structured combination of static analysis using

PEStudio, decompilation using Ghidra, and dynamic debugging using x32dbg, the analysis successfully identified the malware's binary structure, internal code organization, runtime behavior, functional capabilities, and a comprehensive set of indicators of compromise.

The analysis revealed that despite its relatively simple outward appearance as a screen locking tool, ransomware.exe is a sophisticated piece of malware with capabilities spanning system reconnaissance, privilege escalation, process injection, multi-protocol network communication, registry manipulation, and aggressive input blocking. The use of UPX packing to conceal the majority of its import table, combined with the complexity of the AutoIt runtime engine and the malware's ability to bypass API level breakpoints, demonstrates a level of technical sophistication that makes this sample resistant to straightforward analysis.

The findings of this report contribute to a broader understanding of how AutoIt based malware operates and how reverse engineering techniques can be applied to uncover hidden capabilities that would not be visible through behavioral monitoring alone. The indicators of compromise extracted during this investigation provide actionable intelligence that can be used by security teams to detect and respond to infections by this malware family.

CHAPTER 5

CONCLUSION AND RECOMMENDATION

5.1 CONCLUSION

This report has presented a comprehensive reverse engineering analysis of ransomware.exe, a LockScreen ransomware sample compiled using the AutoIt version 3 scripting language and protected with UPX packing. The investigation was conducted entirely within an isolated virtual machine environment using three industry standard tools, PEStudio for static binary analysis, Ghidra for code decompilation, and x32dbg for dynamic debugging, following a structured methodology that progressed from binary structure examination through to capability identification and indicator extraction.

The static analysis phase revealed that the malware was a 32-bit portable executable with an entropy value of 7.875, confirming the presence of UPX compression. Unpacking the binary expanded the file from 445,231 bytes to 792,879 bytes and exposed a dramatically larger import table containing 513 functions, of which 156 were flagged as suspicious. The flagged imports revealed a comprehensive set of malicious capabilities including process injection, multi-protocol network communication over HTTP, FTP, ICMP, and raw sockets, registry manipulation across all major hives, system reconnaissance, and privilege escalation through SeDebugPrivilege and SeShutdownPrivilege.

The decompilation phase in Ghidra identified 2,222 functions within the unpacked binary, with the largest function spanning 30,796 bytes and serving as the primary malicious execution engine. The presence of CreateProcessWithLogonW and LogonUserW within the startup routine confirmed that the malware is capable of creating processes under alternative user credentials, extending its potential for lateral movement within a compromised environment.

The dynamic debugging phase using x32dbg confirmed the malware's runtime behavior, capturing the loading of fourteen system libraries, the hooking of critical window management and input blocking functions, and the successful deployment of the lockscreen payload. The fact that all API level breakpoints recorded zero hits despite the lockscreen executing successfully provided strong evidence of anti-debugging behavior through direct system calls or alternative code execution paths.

Taken together, the findings of this report demonstrate that ransomware.exe is a technically sophisticated piece of malware that leverages the AutoIt scripting platform, UPX packing, and aggressive evasion techniques to deploy a highly effective lockscreen payload while concealing the full extent of its capabilities from casual inspection. The combination of static and dynamic reverse engineering techniques proved essential in uncovering the complete picture of this malware's design and intent.

5.2 RECOMMENDATIONS

Based on the findings of this reverse engineering analysis, the following recommendations are made for organizations seeking to defend against malware of this type and for future analysts approaching similar samples.

From a defensive perspective, organizations should ensure that endpoint security solutions are configured to monitor and alert on the execution of AutoIt compiled binaries, particularly those that have been packed with UPX or exhibit high entropy values. The presence of SeDebugPrivilege or SeShutdownPrivilege in a process token should be treated as a strong indicator of suspicious activity and should trigger immediate investigation. Network monitoring solutions should be configured to detect and alert on ICMP ping sweeps, outbound FTP connections, and connections to unknown HTTP endpoints, all of which are consistent with the network behavior identified in this sample.

Registry monitoring should be implemented across all five major registry hives, with particular attention to modifications to SOFTWARE\Classes\and COM object registrations, which can be used by malware to achieve persistence across system reboots. User awareness training should emphasize the dangers of executing unknown executable files, as LockScreen ransomware of this type is typically delivered through social engineering and requires user interaction to execute.

From an analytical perspective, future analysts approaching AutoIt compiled malware should prioritize unpacking as an early step in the analysis process, as the majority of meaningful indicators are concealed within the compressed payload. The use of entropy analysis and section name inspection in PEStudio provides a reliable and efficient means of identifying packed binaries before proceeding to deeper analysis. When debugging AutoIt compiled binaries with x32dbg, analysts should be aware that the malware may bypass standard

API level breakpoints through direct system calls, and should consider using kernel level breakpoints or memory access breakpoints as an alternative approach.

Finally, the combination of static and dynamic analysis techniques used in this report should be adopted as a standard methodology for malware investigation, as neither approach alone provides a complete picture of a sample's capabilities. Static analysis reveals the full extent of a binary's potential functionality, while dynamic analysis confirms which of those capabilities are actually exercised at runtime and under what conditions. Together they provide the most comprehensive and reliable basis for threat intelligence generation and defensive countermeasure development.